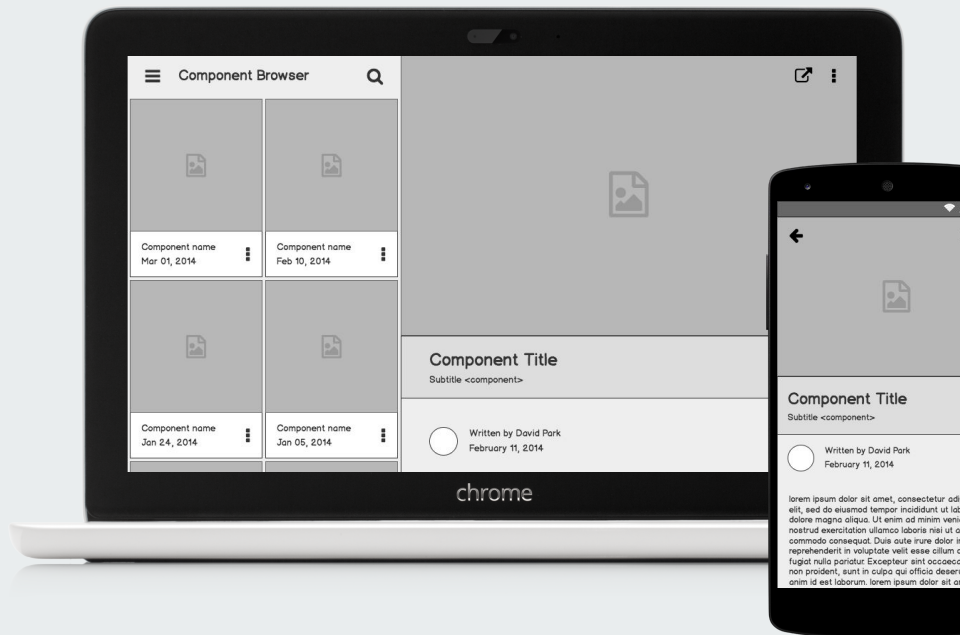




Architecture Challenge

Architecture and Implementation Review



Topics

Objective

Requirements

System Architecture

Architecture Decisions

Implementation Principles

Request Flows

AI Solution

Testing, Deployment and Deliverables



Objective

Build a reliable and maintainable web application for discovering TV series and interacting with their episodes.

The solution should allow users to:

- Search for series using the TVMaze API.
- View series details, seasons, episodes, genres, and summaries.
- Mark episodes as watched.
- Add comments to series and episodes.
- Generate concise AI-powered insights from catalog data and community comments.
- Preserve data across requests while supporting multiple anonymous browser-based identities.



Requirements

(Architecture and code quality)

Architecture and code quality

- Implement the solution using Python.
- Apply Clean Architecture, SOLID principles, and appropriate design patterns.
- Keep business rules isolated from frameworks, databases, and external providers.
- Provide automated tests for the main application behaviors.



Requirements

(Functional scope)

Functional scope

- Search TV series using the TVMaze API.
- Display series details, seasons, episodes, genres, and summaries.
- Persist watched episodes.
- Allow comments on series and episodes.
- Provide AI-generated insights.
- Isolate the AI integration and provide a fallback strategy.



Requirements

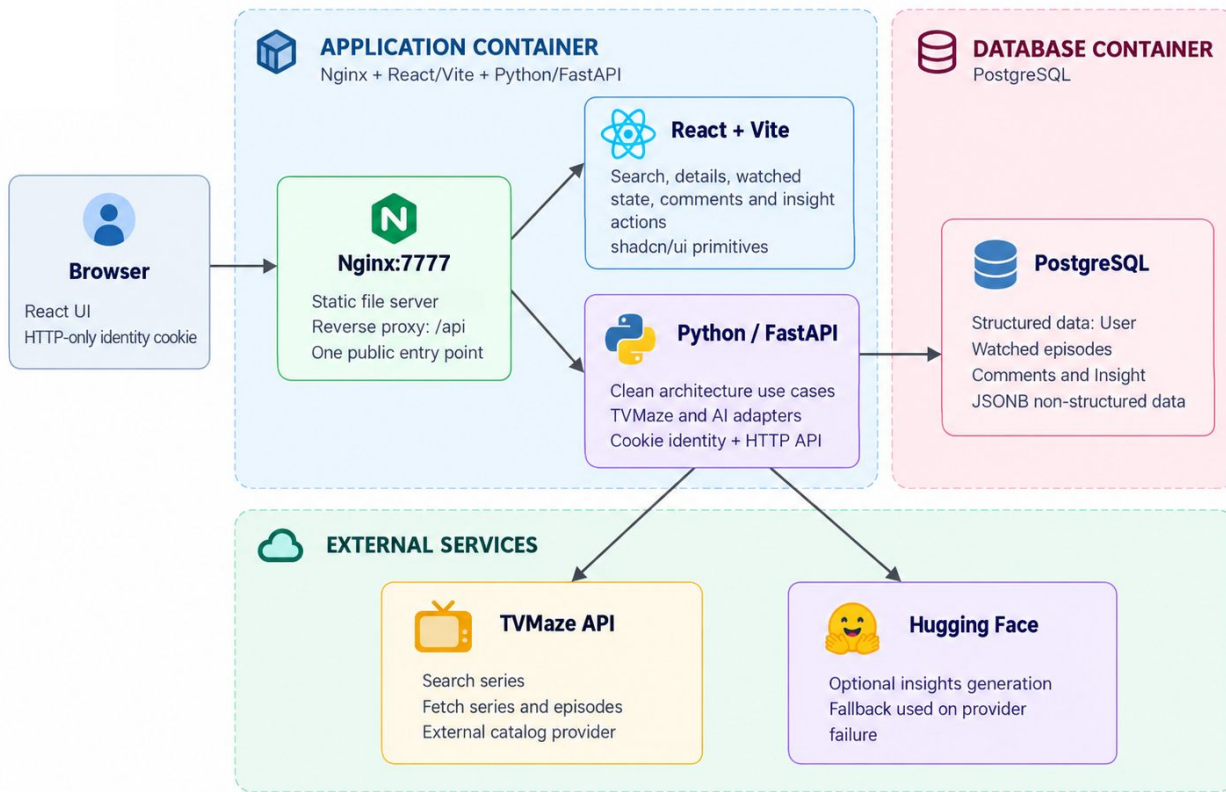
(Deployment and deliverables)

Deployment and deliverables

- Provide a Docker Compose setup with an application container and a database container.
- Expose the application on port **7777**.
- Support one-command startup.
- Include a README, Dockerfile, and Docker Compose configuration.
- Provide a Jenkins-compatible execution flow.
- Deliver a presentation PDF containing the architecture diagram, search sequence, details sequence, and AI explanation.

System Architecture

Containers, frontend, backend, database, and external services



Architecture Decisions

- **FastAPI** -> Chosen as the Python web framework for asynchronous APIs and clear dependency injection.
- **React + Vite** -> Chosen to build a componentized and responsive user interface.
- **shadcn/ui** -> Chosen to provide accessible UI primitives while keeping styling customizable.
- **Nginx** -> Chosen to serve the React build and proxy `/api` requests to FastAPI while preserving the two-container requirement.
- **PostgreSQL** -> Chosen as the database engine because it supports relational data and JSONB storage.
- **SQLAlchemy async + asyncpg** -> Chosen for asynchronous database access and repository isolation.
- **Alembic** -> Chosen to manage database schema migrations.
- **JSONB cache for TVMaze data** -> Chosen to retain external catalog snapshots while keeping structured fields queryable.
- **Hugging Face Inference API** -> Chosen as the AI provider because it is explicitly allowed by the challenge and supports a server-side integration.
- **Deterministic local fallback** -> Chosen to keep the feature available when the AI provider fails, times out, lacks credentials, or returns unusable output.
- **Anonymous browser identity via HTTP-only cookie**
Chosen because authentication was outside the scope, while user-specific watched state and comment authorship still needed to be distinguished.
- **uv dependency management**
Chosen for reproducible and fast Python dependency installation.



Implementation Principles

(Clean Architecture)

Clean Architecture boundaries

Presentation

FastAPI routers, DTOs, HTTP errors, cookie identity



Application

Search, details, watched state, comments, AI insights



Domain

Series, Episode, Comment, WatchedEpisode, Insight



Infrastructure

TVMaze, PostgreSQL, SQLAlchemy, Hugging Face, fallback



Implementation Principles

(S.O.L.I.D.)

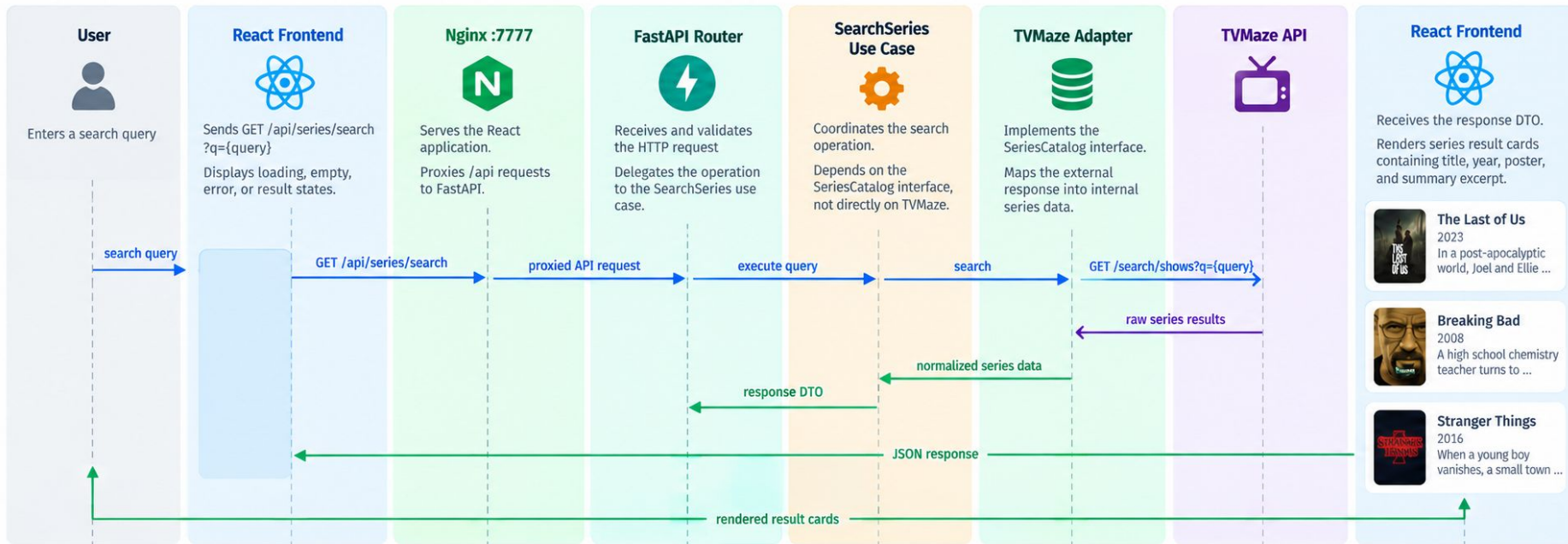
SOLID principles applied

- **Single Responsibility**
Routers handle HTTP, use cases coordinate behavior, and adapters handle external systems.
- **Open/Closed**
New catalog or AI providers can be added through existing interfaces without changing business rules.
- **Liskov Substitution**
Real adapters can be replaced by in-memory fakes or test doubles.
- **Interface Segregation**
Ports are focused on specific capabilities such as catalog access, caching, comments, and insights.
- **Dependency Inversion**
Application services depend on abstractions, not directly on FastAPI, PostgreSQL, TVMaze, or Hugging Face.

The architecture keeps the core testable while allowing infrastructure details to evolve independently.

TV Series Search — Information Flow

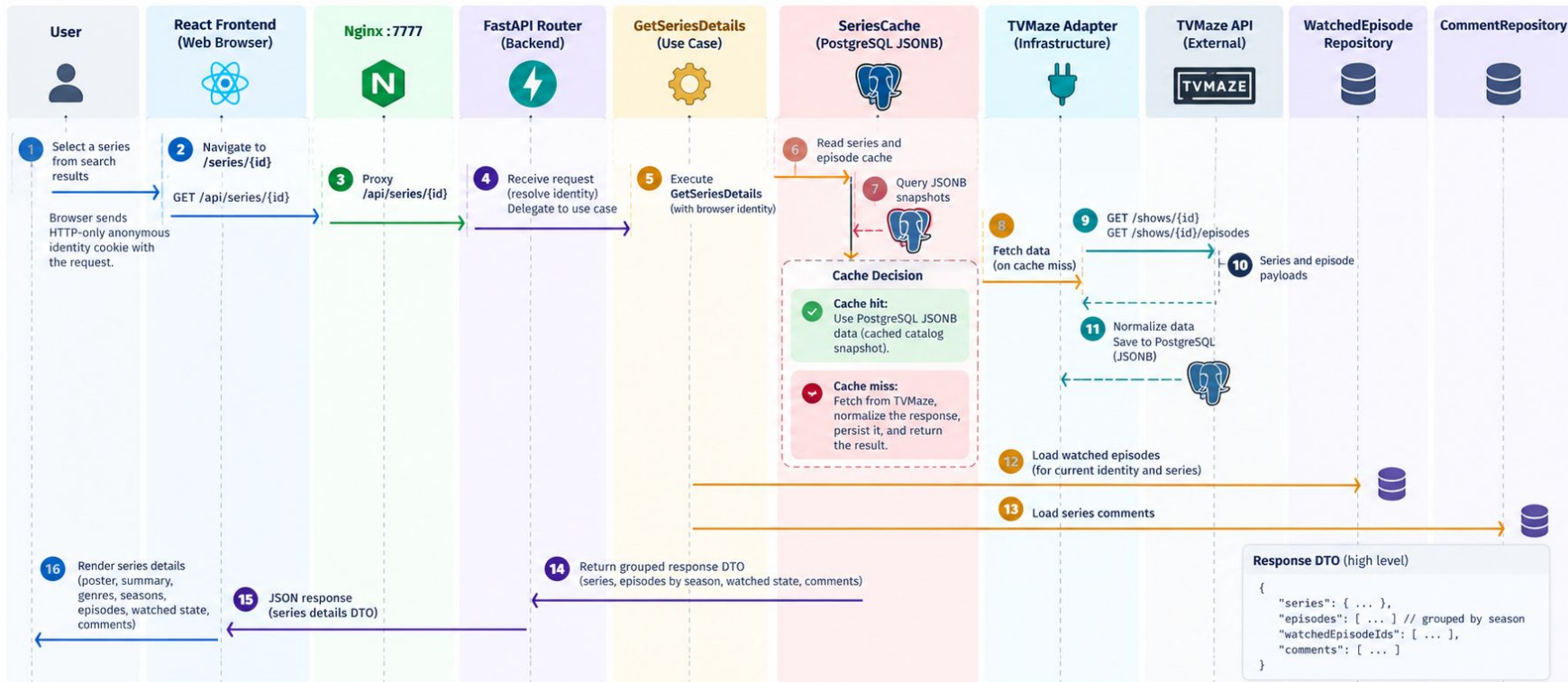
From entering a search query to rendering series results in the frontend.



Series Details — Information Flow

From selecting a series to rendering the full details with episodes, watched state and comments.

The browser sends an HTTP-only anonymous identity cookie with the request to identify the user (per browser).



AI Insight Solution

Optional AI-generated insights for series and episodes, with safe fallback behavior



Input

Information used to generate insights



Series or episode
summary



Genres

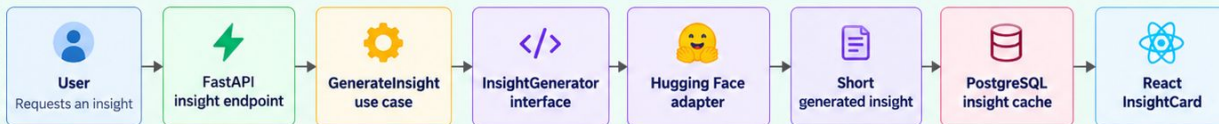


Optional user
comments



Main flow

From user request to generated insight and presentation



Key architectural decisions

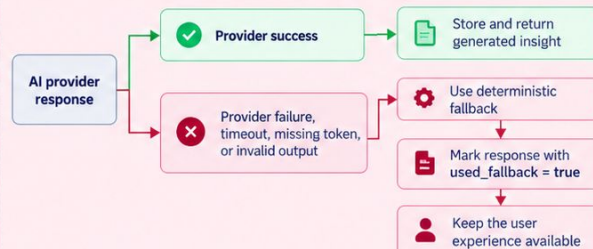
Design principles for a resilient and maintainable solution

- 1 AI integration is isolated behind the `InsightGenerator` interface.
- 2 The application layer does not depend directly on Hugging Face.
- 3 The Hugging Face token remains server-side.
- 4 A fallback provider prevents external AI availability from becoming a single point of failure.
- 5 The same flow supports both series-level and episode-level insights.



Failure strategy

Safe fallback behavior when the AI provider is unavailable



“AI is treated as an optional external capability, not as a dependency of the core business logic.”



Testing

Testing

- Unit tests for domain entities and application use cases.
- Tests for TVMaze response mapping.
- Tests for AI provider fallback behavior.
- Tests for anonymous browser identity.
- API health validation.
- Backend linting with Ruff.
- Frontend production build validation.

Evidence: 9 backend tests passed and successful frontend build.



Deployment

Deployment

- Docker Compose **v3.3**.
- One application container.
- One PostgreSQL database container.
- Application exposed on port **7777**.
- Automatic database migrations on startup.
- PostgreSQL volume for persistence.
- Health check through **/api/health**.
- Single-command startup:



Deliverables

Deliverables

- `Dockerfile`
- `docker-compose.yml`
- Clear `README.md`
- Backend and frontend source code
- Database migrations
- Automated tests
- Architecture documentation
- Architecture and sequence diagrams
- AI integration explanation
- Presentation exported as PDF
- Jenkins-compatible execution flow

The solution is reproducible, testable, containerized, and ready to be executed in a Jenkins-based environment.